Review finale des spécifications par Opus

12 octobre 2026. Modèle exécuté : `claude-opus-5-5`, via Claude Code CLI. Review indépendante du draft 0.1, avant intégration des corrections. Les outils de dépôt étaient désactivés ; aucune inspection du code par ce reviewer n'a eu lieu.

Review finale indépendante — nirs4all-preflight (SPECIFICATION v0.1, ROADMAP, AUDIT)

Opus, 12 octobre 2026. Je n'ai pas pu inspecter les dépôts : le shell n'a renvoyé aucune sortie. Les points qui dépendent du code de DAG-ML sont donc formulés comme des vérifications à faire.

Verdict

Révision ciblée avant PF-00, sans refonte. Le draft intègre correctement les P0 de ma review initiale :

- unités explicites ;
- vues `native` / `canonical` / `supervised` ;
- sonde imbriquée et groupée ;
- abandon du terme SNR et ICC restreint ;
- PERMDISP, sous-échantillonnage sans remise, réseaux sortis de V1 ;
- frontières de dépôts conformes à la doctrine (Methods, IO, Formats, DAG-ML, Core, nirs4all-r, UI sans état, Web).

Il reste trois défauts de niveau P0. Deux réintroduisent de la pseudo-réplication par les chemins par défaut, le troisième laisse indécis le contrat d'exécution entre Preflight, DAG-ML et Methods.

P0 — à corriger avant PF-00

P0-1 — Le voisinage et les permutations par défaut violent l'indépendance (§5.3, §5.5).

- *Défaut :*
  - Par défaut, les voisins excluent seulement les acquisitions du même sample. L'exclusion de l'unité indépendante n'intervient que « si le profil l'exige ».
  - Si un lot partage à la fois une signature spectrale et un niveau de Y, `L_i` mesure surtout l'effet lot.
  - La permutation au niveau sample suppose alors une échangeabilité fausse : la p-value globale est anticonservatrice.
- *Correction :*
  - Le défaut doit exclure les voisins de la même unité indépendante déclarée.
  - Les permutations se font au même niveau.
  - La variante qui inclut le même groupe devient une sensibilité étiquetée.
  - Si aucune unité indépendante n'est déclarée, la statistique globale reste descriptive (`not_identifiable` pour le test).

P0-2 — Les répétitions non déclarées passent silencieusement (§3, tableau « Identité physique inconnue »).

- *Défaut :* l'hypothèse « une acquisition par objet » est enregistrée, puis tous les diagnostics kNN, couverture, doublons et sonde s'exécutent. C'est le cas le plus fréquent dans les jeux réels. On retrouve exactement la cohérence locale triviale décrite dans ma review initiale.
- *Correction :*
  - Lorsque cette hypothèse est active, exécuter un contrôle obligatoire de répétitions présumées : quasi-doublons, Y identique, métadonnées ou horodatages contigus.
  - Publier le taux de paires suspectes.
  - Plafonner à `low` la confiance des résultats fondés sur les voisins et sur la sonde tant que ce taux dépasse un seuil versionné.

P0-3 — L'inversion de contrôle de la sonde et des fournisseurs n'est pas tranchée (§2, §5.6, §8).

- *Défaut :* le §8 dit que « le moteur Rust décide les opérations », alors que le §5.6 attribue l'exécution de la validation à DAG-ML. Deux lectures sont possibles :
  - Preflight pilote la boucle de folds et DAG-ML ne fournit que les splits. Cela viole la règle « orchestration CV = DAG-ML ».
  - DAG-ML exécute la CV imbriquée en appelant lui-même n4m. Dans ce cas, Preflight ne doit consommer que le plan, les OOF et les hashes FIT/EVAL.
- Par ailleurs, d'après mes notes de session (à vérifier au lot 0), la crate Rust DAG-ML n'exposait pas d'API estimateur. DAG-ML est aussi décrit comme « Emscripten » au §8, ce qui reste à confirmer.

- *Correction :* figer en PF-00 la seconde option.
  - `ValidationProvider` reçoit spécification de sonde, plan et budget, et rend OOF, folds et populations.
  - PF-04 dépend explicitement d'une API estimateur DAG-ML → n4m livrée en amont, avec son coût.

P1 — à corriger dans la spec ou la roadmap avant les lots concernés

P1-1 — La passerelle WASM asynchrone impose une machine à états à tout le moteur (§8).

- *Défaut :* la boucle `step → provider_request → provider_response` oblige à rendre reprenables tous les algorithmes qui appellent une PCA : couverture, cohérence locale, 50 PCA de stabilité, permutations. En natif, ces mêmes algorithmes sont synchrones. On aurait donc deux chemins d'orchestration qui divergeront.
- *Correction :*
  - Tester en PF-00 un fournisseur **synchrone** : import wasm-bindgen qui appelle, dans le même worker, l'export Emscripten (copie entre les deux mémoires, sans ASYNCIFY). Les deux modules resteraient séparés.
  - `step()` ne servirait plus qu'à la granularité d'annulation.
  - Garder le protocole asynchrone seulement en repli documenté.

P1-2 — Une seule instance libn4m par processus et par page (§7, §8, §11).

- *Défaut :* plusieurs paquets peuvent chacun embarquer libn4m dans le même hôte :
  - wheel Preflight à côté de la wheel Methods ;
  - `nirs4allpreflight` à côté de `nirs4all-r` ;
  - module WASM Preflight à côté de l'agrégat Core web.
- Les risques sont des conflits de symboles, une divergence de versions (et donc des goldens incohérents) et un double poids sur la page.
- *Correction :* définir par hôte une politique de liaison, consignée dans la matrice de capacités :
  - soit liaison statique avec symboles cachés et version ABI vérifiée au chargement ;
  - soit chargement de la libn4m du paquet Methods ;
  - dans le navigateur, une seule instance Emscripten partagée par Core et Preflight.

P1-3 — La règle `d_95` sur une vue simplement centrée est inadaptée au NIRS brut (§5.3).

- *Défaut :* PC1 capte la diffusion et la ligne de base, donc `d_95` vaut souvent 1 à 3. Couverture, kNN, HDBSCAN et PERMANOVA opèrent alors dans un espace dominé par la nuisance. Différer l'analyse parallèle ne règle pas le défaut par défaut.
- *Correction :*
  - Prendre comme défaut une règle sans Y robuste à cette domination, par exemple l'analyse parallèle de Horn par permutation des colonnes, bornée à [3, 20].
  - Garder `d_95` et la vue SNV comme sensibilités publiées.
  - Signaler `dominated_by_first_component` lorsque $\lambda_1/\Sigma\lambda$ dépasse un seuil versionné.

P1-4 — Le nombre de permutations B n'est pas fixé (§6, §10).

- *Défaut :* aucun B ni nombre de réplicats par défaut n'est spécifié, donc le plan ne peut pas chiffrer les coûts. Avec `(1+b)/(1+B)` et la correction de Holm sur m tests, une famille peut devenir non significative par construction.
- *Correction :*
  - Fixer B par protocole, par exemple 999, et imposer $B \geq m/\alpha$.
  - Sinon, rendre `insufficient_resolution`.
  - Inclure B dans le budget calculé par `plan()`.

P1-5 — Budgets mémoire WASM (§10).

- *Défaut :* la mémoire linéaire WASM ne rétrécit jamais. Le pic réel est la somme des maxima atteints par les deux modules, pas un instantané. Les 512 Mio du profil étendu sont irréalistes avec 128 Mo de X et des copies inter-modules.
- *Correction :*
  - Budgéter le maximum atteint par chaque module.
  - Recréer le worker entre modules lourds, comme stratégie déclarée.
  - Réduire le profil étendu au navigateur seulement après mesure en PF-00.

P1-6 — Chiffrage (ROADMAP §1–3).

- *Défaut :*
  - Les demandes amont n'ont pas de charge : PCA diagnostique dans Methods, API estimateur et fournisseur DAG-ML, adaptateur IO, contrats UI. Elles conditionnent pourtant le chemin critique.
  - PF-01, à 10–14 j, est irréaliste : cinq bindings, MEX, toolchain Rust du package R et wasm-bindgen.
- *Correction :*

- Ajouter une ligne de coût par dépôt amont, avec sa release dans la cohorte.
- Porter PF-01 à environ 16–22 j.
- Recalculer l'enveloppe totale, qui dépassera probablement 130–190 j-p.

P2 — précisions

- **§5.3, `D_eff`.** Il est calculable exactement sans spectre complet : $\Sigma\lambda = \mathrm{tr}(S)$ et $\Sigma\lambda^2 = \|XcXc^\top\|^2_F/(n-1)^2$, via la matrice de Gram n×n. C'est une statistique propre au diagnostic, à calculer en Rust. Supprimer la variante « borne ».
- **§3, agrégation.** Une moyenne sur r_i acquisitions inégaux a une variance s_r^2/r_i. Les samples très répétés paraissent plus centraux et moins bruités. Publier la dispersion de r_i et proposer un équilibrage déclaré (sous-échantillonnage à r_min) en sensibilité.
- **§5.5, plafond de `L_i`.** Avec une précision de référence σ_ref, L ne peut dépasser $1 - \sigma_ref^2/s_y^2$. Publier ce plafond plutôt qu'une tolérance arbitraire.
- **§5.6, définition du R^2 et agrégation des prédictions.**
  - Définir $R^2_OOF = 1 - SSE_modèle / SSE_baseline$ avec la moyenne du train de chaque fold, et non avec la moyenne globale, qui fuit.
  - Préciser si les prédictions par acquisition sont moyennées par sample ou si les spectres le sont avant prédiction.
- **§5.7, `min_cluster_size` sous sous-échantillonnage.** À 80 %, il doit être soit mis à l'échelle, soit fixé, mais déclaré. Les densités changent sinon de façon systématique.
- **§5.8, prédiction des métadonnées.** Grouper par le niveau supérieur qui contient le facteur (lot, campagne). Sans cela, la prédiction de l'instrument reste triviale.
- **§6, RNG.** Le générateur doit être implémenté dans le core Rust (algorithme figé), indépendant de n4m et du runtime hôte. Les seeds de folds DAG-ML se dérivent de la même racine.
- **§7, R et MATLAB.**
  - R : préciser la stratégie Rust (crates vendorisées et cargo, ou lib précompilée ; Rtools ne fournit pas cargo) et sa cible R-universe.
  - MATLAB : tester les conflits de libstdc++ propres à MATLAB sous Linux, causés par le C++ de libn4m (libstdc++ statique ou ABI C stricte).
- **§1 contre ROADMAP, cohérence MVP.**
  - L'ICC équilibré est admis au MVP (§5.2), mais absent de PF-02.
  - Le MVP (PF-03) exige des permutations groupées valides alors que leur infrastructure n'arrive qu'en PF-06.
  - Déplacer le moteur de permutation vers PF-03, ou marquer la cohérence locale comme descriptive au MVP.

Limites produit

Les exclusions sont saines :

- pas de score global ;
- pas de suppression automatique ;
- pas de calcul en JS ;
- Quality reste distinct ;
- Lab et Drafts sont exclus.

Il faut cependant éviter que la page web présente une démo bornée comme une qualification. Chaque export doit porter le profil exact (complet ou partiel) et les modules indisponibles, ce que le §9 prévoit déjà. Ajouter également ce marquage aux composants UI eux-mêmes, par une prop obligatoire `profileStatus`.

Verdict final

Accepté sous conditions. Prérequis bloquants :

- P0-1 à P0-3 intégrés dans la spec avant PF-00 ;
- P1-1, P1-2 et P1-3 convertis en critères de sortie de PF-00 ;
- P1-6 répercuté dans l'enveloppe de la roadmap.

Les P2 peuvent être traités au fil des lots, sans bloquer le démarrage.